

Noct Finance V1 — Complete Architecture

34-document consolidated implementation specification

1. Executive Summary

Purpose

Noct Finance V1 is a private DeFi lending application built using the current Horizen Vela architecture as the confidential execution substrate, with Horizon as the blockchain/settlement environment and UltraHonk + zkVerify as the ZK proof layer.

V1 intentionally has a small market:

| Role | Asset |

|---|---|

| Deposit | USDC |

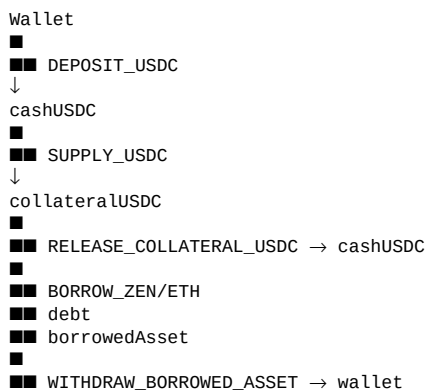
| Supply/collateral | USDC |

| Borrow | ZEN, ETH |

| ZEN/ETH liquidity | Funded by Noct for the testnet |

| User position | Private |

Core state lifecycle



Critical semantic decisions

- Deposit is not supply.
- Released collateral returns to private cash first.
- Cash withdrawal is the operation that externally withdraws USDC.
- Borrow creates private debt and a private borrowed-asset balance.
- Borrowing does not automatically send the asset to the wallet.
- Withdrawing a borrowed asset does not reduce debt.
- Private collateral, debt, LTV and health factor must not be exposed as public Noct state.

System principle

Noct is one logical state machine. Vela, the ZK layer, zkVerify and Horizon are supporting execution/verification/settlement layers around that state machine.

Completion standard

A feature is complete only when state semantics, security, privacy, failure recovery, tests and required performance evidence are present.

2. Design Objectives

Primary objectives

Privacy

Prevent an observer from reliably linking a wallet to its private Noct position and learning:

- collateral;
- debt;
- LTV;
- health factor;
- cash balance;
- borrowed balance;
- private transaction history.

Security

Every state-changing transition MUST be:

- authenticated;
- deterministic;
- versioned;
- replay-protected;
- atomic;
- invariant-preserving.

Speed

No ZK component may enter a user-facing critical path without measurement. Proof generation is treated as a likely bottleneck and MUST be benchmarked independently from verification.

Simplicity

V1 excludes:

- multi-collateral;
- permissionless lender liquidity;
- E-mode;
- flash loans;
- credit delegation;
- sophisticated rate curves;
- recursive proofs.

Engineering priority

correctness → security → privacy → recovery → performance → UX optimization

Speed must not be achieved by weakening an invariant.

3. Vela Integration Architecture

Current Vela fit

The current Vela repository describes a Secure Processor Manager and a WASM Executor. The Executor uses Wasmtime-go and runs the WASM application inside an AWS Nitro Enclave in the intended architecture. Vela supports isolated application state/funds and blockchain coordination.

Noct MUST be implemented as a Vela WASM application rather than creating a second private execution platform.

Architecture

```
flowchart TD
    Chain[Horizon / ProcessorEndpoint] --> Manager[Vela Manager]
    Manager --> Executor[WASM Executor / Nitro Enclave]
    Executor --> Noct[Noct WASM]
    Noct --> State[Encrypted Versioned State]
    State --> Noct
    Noct --> Executor
    Executor --> Manager
    Manager --> Chain
```

Coding structure

Use the current Vela private-transfer pattern:

```
runtime/
main.go
app/
state/
transitions/
risk/
interest/
oracle/
commitments/
events/
serialization/
```

main.go MUST remain a thin WASM bridge. Business logic belongs in app/.

Vela responsibilities

Vela handles request coordination, WASM execution, private state persistence, events and blockchain-facing processing.

Noct handles lending semantics, accounting, risk and transition validation.

Important implementation rule

Use the exact Vela commit/tag selected for the project as the source of truth. Do not assume that an older Vela document describes the current runtime.

4. Horizon Architecture

Role

Horizon is the public blockchain/settlement boundary. It MUST NOT become a plaintext database of user lending positions.

Public layer

Publicly visible data may include:

- application identity;
- state commitments;
- proof/receipt references;
- configuration identifiers;
- aggregate protocol statistics;
- external settlement transactions.

Private layer

Private data includes:

- user cash;
- user collateral;
- debt;
- borrowed balance;
- LTV;
- health factor;
- user-specific history.

Custody flow

```
User wallet
↓
Vela ProcessorEndpoint / application funds
↓
Noct private accounting
```

For a normal USDC withdrawal:

```
private cashUSDC
↓
authorized withdrawal
↓
Vela settlement
↓
Horizon
↓
wallet
```

Important limitation

A public blockchain transfer can expose recipient and amount. Noct therefore claims private lending state, not absolute blockchain anonymity.

Developer rule

Never emit user position values from Solidity or public application events.

5. Privacy Model

Privacy goal

Noct protects the relationship between a public wallet and the user's private lending position.

Sensitive fields

```
cashUSDC
collateralUSDC
borrowedZEN
borrowedETH
debtZEN
debtETH
LTV
healthFactor
private transaction history
account → position linkage
```

Privacy boundaries

```
flowchart LR
  Wallet[Public Wallet] --> Request[Encrypted Request]
  Request --> TEE[Vela Confidential Execution]
  TEE --> State[Private Position]
  State --> Commit[Public Commitment]
  State --> Proof[ZK Proof]
  Proof --> Verify[zkVerify]
```

Leakage sources

Review:

- public events;
- settlement timing;
- withdrawal amount;
- recipient address;
- proof public inputs;
- facilitator logs;
- browser logs;
- analytics;
- RPC traces.

Hard rule

The frontend, facilitator and prover MUST NOT log private financial values.

V1 claim

The core privacy property is private position state and reduced linkage between public chain activity and individual lending positions. External settlement may remain observable.

6. Trust Model

Trust domains

wallet authorization
↓
Vela request security
↓
Nitro TEE confidentiality
↓
Noct deterministic state machine
↓
ZK proof
↓
zkVerify
↓
Horizon settlement

TEE assumptions

Production confidentiality relies on the Nitro Enclave deployment, attestation, enclave measurement and supply-chain integrity. Local Docker execution is not evidence of production enclave security.

ZK assumptions

The proof system establishes the correctness of the specific transition statement. zkVerify independently verifies the proof.

Facilitator

A facilitator is a relayer/gas payer. It MUST NOT be treated as the source of truth for balances.

Frontend

Frontend checks are advisory only. All financial rules MUST be enforced in the protocol state machine and/or proof.

Oracle

Oracle correctness is a system dependency. Stale/missing oracle inputs must fail closed for risk-sensitive operations.

7. Noct Account / Identity Model

Goal

Separate public wallet identity from private Noct position identity.

Vela client interaction

The current Vela TypeScript client derives a P-521 communication key from wallet signing material and uses it for encrypted communication and encrypted user events. Reuse that client flow.

Conceptual model

```
wallet signer
↓
Vela communication key
↓
Noct account identity
↓
position commitment
↓
private account state
```

Account state

```
Account {
  accountCommitment
  positionCommitment
  nonce
  status
}
```

Initialization

1. Connect wallet.
2. Derive Vela communication key.
3. Register/associate the key as required by Vela.
4. Initialize private Noct account.
5. Create initial zero-state commitment.

Storage rule

Do not put raw private keys or seed phrases in localStorage. Use wallet signing and Vela's documented Web Crypto/key derivation flow.

8. NoctState V1

Logical state

```
NoctStateV1
■■■ GlobalState
■■■ ReserveState
■■■ OracleState
■■■ ProtocolConfig
■■■ CommitmentState
■■■ PrivateAccountState
```

Private account

```
PrivateAccount {
  cashUSDC
  collateralUSDC

  borrowedZEN
  borrowedETH

  debtZEN
  debtETH

  positionNonce
  pendingSettlements
}
```

Global/public state

```
GlobalState {
  protocolVersion
  stateVersion
  configCommitment
  oracleCommitment
  positionRoot
  stateCommitment
}
```

Derived values — do not store

- LTV;
- health factor;
- current USD debt;
- borrowing capacity;
- liquidation status.

Calculate these from current state, oracle and config.

Numeric rule

Use integer/fixed-point arithmetic. No float/double in financial logic.

9. State Commitment Model

Purpose

A commitment binds private state to a public cryptographic reference without exposing the private values.

Hierarchy

```
NoctStateRoot
■■■ GlobalRoot
■■■ ReserveRoot
■■■ ConfigRoot
■■■ OracleRoot
■■■ PositionRoot
```

Conceptual commitment

```
stateCommitment =
H(
domain,
appId,
protocolVersion,
stateVersion,
account/position commitments,
reserve commitment,
config commitment,
oracle commitment
)
```

Canonical encoding

Do not hash arbitrary JSON text. Use fixed field order and canonical integer/byte encoding shared across Go, Noir, TypeScript and contracts where applicable.

Domain separation

At minimum distinguish:

```
NOCT_STATE_V1
NOCT_TRANSITION_V1
NOCT_WITHDRAWAL_V1
NOCT_LIQUIDATION_V1
```

Tests

Create golden vectors for state and transition commitments before connecting ZK circuits.

10. Protocol Configuration

Configuration model

Noct must have one canonical versioned configuration rather than separate risk values hard-coded in frontend, WASM and circuits.

```
ProtocolConfig
■■■ assetConfig
■■■ riskConfig
■■■ interestConfig
■■■ oracleConfig
■■■ reserveConfig
■■■ zkConfig
```

V1 asset rules

```
collateralAsset = USDC
borrowAssets = [ZEN, ETH]
```

Risk parameters

Configuration supports:

- maximum LTV;
- liquidation threshold;
- liquidation bonus;
- close factor.

The architecture discussion has used 75% as the V1 borrowing-factor target and 80% as a candidate liquidation threshold. Exact production numeric parameters MUST be frozen before deployment and MUST NOT be guessed in code.

Configuration commitment

```
configCommitment = H(canonicalConfig)
```

Every proof binds to this commitment.

11. Asset & Reserve Model

V1 roles

| Asset | Deposit | Supply | Borrow | V1 role |

|---|---|---|---|---|

| USDC | yes | yes | no | collateral |

| ZEN | no normal V1 deposit | no | yes | borrow asset |

| ETH | no normal V1 deposit | no | yes | borrow asset |

Reserve structure

```
ZENReserve {
  availableLiquidity
  totalDebtPrincipal
  borrowIndex
  lastAccrualTimestamp
}
```

```
ETHReserve {
  availableLiquidity
  totalDebtPrincipal
  borrowIndex
  lastAccrualTimestamp
}
```

Noct-funded reserves

The test application is funded by Noct for ZEN and ETH liquidity. Reserve exhaustion must be tested.

Critical distinction

```
reserve liquidity != borrowed balance != debt
```

If Alice borrows and then withdraws 20 ZEN:

```
borrowedZEN = 0
debtZEN = 20
```

Debt persists.

12. Complete State Machine

Transition set

```
T01 DEPOSIT_USDC
T02 WITHDRAW_CASH_USDC
T03 SUPPLY_USDC
T04 RELEASE_COLLATERAL_USDC
T05 BORROW_ZEN / BORROW_ETH
T06 WITHDRAW_BORROWED_ASSET
T07 REPAY_ZEN / REPAY_ETH
T08 LIQUIDATE
```

Universal pipeline

```
flowchart TD
R[Request] --> A[Authenticate]
A --> V[Validate state/version]
V --> P[Proof/authorization]
P --> C[Compute transition]
C --> I[Check invariants]
I --> N[New state]
N --> S[Settlement if required]
```

Core semantics

Deposit

cashUSDC += amount

Supply

cashUSDC -= amount; collateralUSDC += amount

Release collateral

collateralUSDC -= amount; cashUSDC += amount

Borrow

debt += amount; borrowedAsset += amount; reserve -= amount

Borrowed withdrawal

borrowedAsset -> settlement; debt unchanged.

Cash withdrawal

cashUSDC -> settlement.

Repay

Debt decreases.

Liquidation

Debt/collateral change atomically according to the finalized risk rules.

13. ZK Architecture

Principle

ZK is used to prove private transition correctness; Vela supplies confidential execution.

```
flowchart LR
    State[Private State] --> Witness[Witness Builder]
    Witness --> Prover[UltraHonk Prover]
    Prover --> zk[zkVerify]
    zk --> Authorize[Transition Authorization]
    Authorize --> Vela[Vela / Noct WASM]
```

Do not build one giant circuit

Use focused circuits for:

- supply;
- borrow;
- repay;
- release collateral;
- withdrawal;
- liquidation.

ZK verification abstraction

```
interface ZKVerifier {
    verifyTransition(context: VerificationContext): Promise<VerificationResult>;
}
```

Implement both:

- direct/verify-only;
- aggregation-receipt.

Performance gate

No ZK path becomes mandatory in a user-facing critical path until it passes the benchmark defined in document 28.

14. UltraHonk Circuit Architecture

Tooling baseline

Current zkVerify documentation supports Noir UltraHonk and documents:

- supported V3_0/V0_84/Legacy verifier variants;
- maximum 32 public inputs;
- maximum evaluation domain size 2^{25} .

Pin exact versions before deployment.

Layout

```
circuits/  
common/  
commitments.nr  
arithmetic.nr  
membership.nr  
replay.nr  
supply/main.nr  
borrow/main.nr  
repay/main.nr  
release/main.nr  
withdrawal/main.nr  
liquidation/main.nr
```

Public inputs

Keep small and intentional:

- app/domain;
- state commitment;
- config commitment;
- oracle commitment;
- transition identifier;
- replay/nonce commitment;
- resulting state commitment.

Private witness

- private account;
- balances;
- collateral;
- debt;
- borrowed amount;
- membership witness;
- private nonce/randomness as required.

Circuit rules

No floating point. Test overflow, underflow, range and boundary conditions.

15. zkVerify Integration

Role

zkVerify verifies Noct UltraHonk proofs and can expose aggregation receipts for on-chain consumption.

Verify-only path

Proof → zkVerify → verification result → Noct coordinator

Aggregated path

Proof → zkVerify → aggregation → receipt/Merkle path → Noct/Horizon verifier

Frozen architecture decision

Production target:

> zkVerify aggregation receipt consumption.

Condition:

> only after measuring total latency.

Fallback:

> direct/verify-only consumption for testnet or a transition whose aggregation path fails the performance gate.

Binding

Proof context should bind:

- app ID;
- chain ID;
- circuit version;
- VK hash;
- config commitment;
- state commitment;
- transition nonce/type.

Integration advice

Do not make the core state machine depend on a zkVerify SDK object. Wrap the provider behind the Noct ZKVerifier interface.

16. Vela TEE Execution

Execution model

Current Vela uses:

- Secure Processor Manager;
- WASM Executor;
- Wasmtime-go;
- encrypted state;
- AWS Nitro Enclave in the intended production environment.

Noct execution

```
vela request
↓
WASM bridge
↓
transition dispatcher
↓
state load
↓
risk/interest logic
↓
state mutation
↓
commitment/result
↓
event/settlement instruction
```

Thin bridge

Use `main.go` only for:

- pointer/byte conversion;
- calling application functions;
- serializing the result.

Lending logic belongs in `app/`.

Determinism

Noct guest logic must not rely on arbitrary external HTTP, random execution, or uncontrolled time. Required context must be authenticated and deterministic.

17. Settlement Architecture

Principle

Internal private accounting and external token settlement are separate states that must reconcile.

Withdrawal lock

```
available balance
↓
lock amount
↓
pending settlement
/ \
success failure
↓ ↓
consume unlock
```

Cash USDC

cashUSDC → lock → Vela withdrawal → wallet

Borrowed ZEN/ETH

borrowedZEN → lock → Vela withdrawal → wallet

Debt does not change when the borrowed asset is withdrawn.

Idempotency

Settlement uses an immutable settlement ID/request ID. Repeating the same request must never create a second transfer.

Failure rule

A failed settlement unlocks the balance and leaves the economic position intact.

18. Oracle Architecture

Required prices

V1 risk calculations need:

- USDC;
- ZEN;
- ETH.

Snapshot

```
OracleSnapshot {  
  epoch  
  timestamp  
  usdcPrice  
  zenPrice  
  ethPrice  
  commitment  
}
```

Flow

```
flowchart LR  
  Source[Approved price source(s)] --> Snapshot[Authenticated snapshot]  
  Snapshot --> Vela[Vela]  
  Snapshot --> Circuit[ZK witness/commitment]  
  Vela --> Risk[Noct risk engine]
```

Rules

- A borrower/liquidator MUST NOT supply an arbitrary price.
- Risk-sensitive transitions MUST reject stale oracle data.
- The exact freshness threshold is configuration.
- If no valid price is available, the safe response is fail-closed.

Privacy

The market price can be public. The user's quantity multiplied by that price remains private.

19. Interest Accrual

V1 design

Use a reserve-level borrow index instead of continuously updating every user.

```
Reserve
borrowIndex
lastAccrualTimestamp
configuredBorrowRate
```

A user's current debt is derived conceptually as:

```
currentDebt =
principal
× currentBorrowIndex
÷ entryBorrowIndex
```

V1 rate model

Start with a simple protocol-configured rate per borrow asset. Do not introduce a complex utilization/kink curve until the baseline system is correct and benchmarked.

Arithmetic

All rates/indexes use fixed-point integers. Define:

- scale;
- overflow bounds;
- rounding direction;
- minimum/maximum accrual interval.

Lazy accrual

Update an asset's index when a transition needs current debt:

- borrow;
- repay;
- liquidation;
- risk-sensitive state changes.

Do not perform account-by-account accrual every block.

20. Liquidation Discovery

Problem

Liquidators need to find unsafe positions without a public list of health factors.

V1 direction

Use dedicated infrastructure that operates from the confidential execution boundary or privacy-preserving candidate proofs.

```
flowchart TD
  Oracle --> Detector[Liquidation Detector]
  Detector --> Candidate[Private Candidate]
  Candidate --> Prover[Dedicated Prover Worker]
  Prover --> zk[zkVerify]
  zk --> L[Liquidation Transition]
  L --> V[Vela]
```

Security boundary

A discovery worker may identify a candidate, but the liquidation transition itself MUST independently verify:

- current state;
- current oracle snapshot;
- configuration;
- liquidation predicate;
- liquidation amount.

Performance

Liquidation MUST NOT depend on the borrower's browser.

Open item

Exact private candidate-discovery mechanism must be validated against the selected Vela version before production freeze.

21. Transaction / Request Lifecycle

Request states

```
CREATED
SUBMITTED
PROCESSING
PROOF_PENDING
VERIFIED
EXECUTING
SETTLEMENT_PENDING
COMPLETED
FAILED
```

Client flow

The current Vela client pattern is:

1. wallet connection;
2. P-521 key derivation;
3. key association;
4. payload encryption;
5. request submission;
6. completion event;
7. encrypted user-event decryption.

Request envelope

```
type NoctRequest = {
  requestId: string;
  appId: bigint;
  transitionType: number;
  encryptedPayload: Uint8Array;
  stateVersion: bigint;
  nonce: bigint;
  proofReference?: string;
};
```

UI

Show distinct stages:

- preparing;
- proving;
- verifying;
- executing;
- settling;
- completed.

Never show “completed” while canonical state is pending.

22. Failure & Recovery

Failure classes

- client;
- network;
- proof;
- zkVerify;
- aggregation;
- Vela;
- persistence;
- oracle;
- settlement;
- replay/stale-state.

Atomicity

A failed state transition MUST leave the canonical state unchanged.

Settlement recovery

```
balance
↓ lock
pending
■■■ success → consume
■■■ failure → unlock
```

Retry

Retries use immutable IDs. A network timeout is not proof of failure; reconcile before creating a second state mutation.

Reconciliation worker

Compare:

- pending settlement state;
- Vela completion events;
- Horizon receipts.

The worker is not allowed to invent financial state.

23. Replay Protection

Threat

An attacker copies an accepted request/proof and submits it again.

Required binding

Every transition should bind:

```
application ID
chain ID
protocol version
state version/root
position/account commitment
transition type
nonce
asset
config commitment
circuit/VK version
```

Nonce

```
nonce = 7
request proves nonce == 7
success → nonce = 8
```

State root

A proof bound to an old state root must fail after that position/state changes.

Nullifiers

Use nullifiers only where the transition has a specific need. Do not add a global nullifier system automatically.

Cross-context replay

The proof domain must prevent use in another Noct app, chain or protocol version.

24. Privacy Leakage Analysis

Leakage channels

Public chain

- sender;
- event timing;
- settlement recipient;
- settlement amount;
- proof metadata.

Vela

- plaintext events;
- errors;
- debug output.

Backend

- prover logs;
- facilitator logs;
- metrics;
- crash reports.

Client

- analytics;
- console logs;
- local storage;
- telemetry.

Controls

- encrypted user payloads;
- encrypted user events;
- opaque IDs;
- minimal proof public inputs;
- secret redaction;
- no financial analytics;
- short-lived sensitive buffers.

Timing

Perfect timing privacy is not guaranteed. Analyze transition-dependent timing and avoid unnecessary public timing signals.

External settlement

Normal public token settlement can reveal some information. The implementation must state this limitation accurately.

25. Threat Model

Adversaries

- passive blockchain observer;
- malicious borrower;
- malicious liquidator;
- malicious facilitator;
- compromised frontend;
- compromised RPC provider;
- oracle manipulator;
- replay attacker;
- malicious infrastructure operator;
- invalid-proof submitter.

Protected assets

- user funds;
- protocol reserves;
- private position state;
- debt/collateral correctness;
- proof integrity;
- settlement integrity.

Control mapping

| Threat | Control |

|---|---|

| Replay | state root + nonce + domain |

| Double withdrawal | pending lock + settlement ID |

| Invalid borrow | risk check + proof |

| Oracle spoof | authenticated snapshot |

| Facilitator abuse | signed request + encrypted payload |

| Wrong circuit | VK/circuit version binding |

| Stale state | state-version/root check |

Every real incident becomes a regression test.

26. Security Model

Defense in depth

flowchart TD
Wallet --> Auth[Request Auth]
Auth --> Replay[Replay Protection]
Replay --> TEE[Vela Confidential Execution]
TEE --> Risk[Noct Risk Engine]
Risk --> Proof[UltraHonk]
Proof --> ZK[zkVerify]
ZK --> Commit[Atomic State Commit]
Commit --> Settle[Settlement]
Settle --> Recon[Reconciliation]

Security rules

- Frontend never enforces the final financial rule.
- Facilitator never becomes financial source of truth.
- Vela execution is deterministic.
- ZK proof must bind to the correct protocol state/config.
- Settlement requires an authorized internal balance.
- All withdrawals are idempotent.

High-priority audit areas

1. fixed-point arithmetic;
2. commitments;
3. replay/version logic;
4. withdrawal locks;
5. oracle;
6. liquidation;
7. proof-to-state binding.

27. Performance Model

End-to-end latency

request construction
↓
witness generation
↓
UltraHonk proving
↓
zkVerify verification
↓
optional aggregation
↓
receipt availability
↓
Vela execution
↓
Horizon settlement

Classes

Interactive:

- supply;
- repay;
- release;
- withdrawal.

Risk-critical:

- borrow;
- liquidation.

Background:

- proof worker;
- aggregation;
- reconciliation.

Required measurements

For every ZK transition:

- p50;
- p95;
- p99;
- max;
- queue time;
- CPU;
- RAM;
- proof size;
- circuit constraints;
- failure/retry rate.

Architectural conclusion

Browser proving is not a V1 hard dependency. Use dedicated proving infrastructure unless benchmark evidence shows browser proving is acceptable.

28. Benchmarking Plan

Goal

Benchmark before making a ZK path a mandatory critical path.

Harness

```
benchmark/  
circuits/  
witness/  
prover/  
zkverify/  
aggregation/  
vela/  
horizon/  
reports/
```

Matrix

| Operation | Proof | Verify | Aggregate | Receipt | E2E |

|---|---|---|---|---|---|

| Supply | ✓ | ✓ | ✓ | ✓ | ✓ |

| Borrow | ✓ | ✓ | ✓ | ✓ | ✓ |

| Repay | ✓ | ✓ | ✓ | ✓ | ✓ |

| Release | ✓ | ✓ | ✓ | ✓ | ✓ |

| Borrowed withdrawal | ✓ | ✓ | ✓ | ✓ | ✓ |

| Liquidation | ✓ | ✓ | ✓ | ✓ | ✓ |

Measurements

Record:

- exact circuit commit;
- exact Noir/bb versions;
- Vela commit;
- hardware;
- proof time;
- verification time;
- aggregation delay;
- receipt delay;
- Vela processing;
- Horizon confirmation.

Use p50/p95/p99/max, not averages alone.

Go/no-go

A path passes only if correctness, security, resource usage and latency all meet the agreed engineering criteria.

29. Testing Strategy

Test pyramid

Unit
↓
Property
↓
Circuit
↓
Vela integration
↓
Horizon integration
↓
E2E
↓
Security/performance

Core properties

deposit creates cash only
supply moves cash → collateral
release moves collateral → cash
cash withdrawal cannot exceed cash
borrow cannot exceed liquidity
borrow cannot exceed risk limit
borrowed withdrawal does not reduce debt
repay reduces debt
replay cannot mutate state twice
failed settlement restores spendable balance

E2E flow

deposit 100 USDC
→ supply 100
→ borrow ZEN
→ withdraw borrowed ZEN
→ repay
→ release collateral
→ withdraw cash

Regression

Every security bug gets a test and a threat-model entry.

30. End-to-End Architecture

Full system

```
flowchart TB
    U[User] --> W[Wallet]
    W --> UI[Noct Web App]
    UI --> VTS[Vela TypeScript Client]
    UI --> PR[Proof Worker]
    PR --> ZK[zkVerify]
    VTS --> PE[ProcessorEndpoint]
    PE --> VM[Vela Manager]
    VM --> EX[WASM Executor / Nitro Enclave]
    EX --> N[Noct WASM]
    N --> PS[Private State]
    N --> PE
    PE --> H[Horizon]
    ZK --> AUTH[Proof/Receipt Authorization]
    AUTH --> N
```

Borrow example

```
wallet authorization
→ encrypted request
→ proof generation
→ zkVerify
→ Vela execution
→ private state update
→ debt + borrowed balance
→ optional borrowed-asset withdrawal
→ public settlement
```

System feedback

```
oracle → risk → borrow/liquidation
benchmark → ZK placement
security → invariants → tests
Vela capability → implementation constraints
```

31. Deployment Architecture

Environments

local → testnet → production

Local

Use the Vela Starter Kit local environment for:

- WASM;
- contracts;
- request lifecycle;
- integration testing.

Local execution does not prove production Nitro security.

Testnet

Use:

- target Horizon testnet;
- deployed Vela;
- zkVerify test environment;
- controlled Noct ZEN/ETH reserves;
- approved test USDC.

Artifacts to pin

Vela commit
WASM hash
circuit hash
VK hash
config hash
contract addresses
chain ID
toolchain versions

Secret management

Never commit:

- private keys;
- cloud credentials;
- facilitator keys;
- RPC credentials;
- user secret material.

32. V1 Limitations

Scope

V1 intentionally has:

- one collateral asset (USDC);
- two borrow assets (ZEN, ETH);
- protocol-funded borrow liquidity;
- limited risk model;
- limited liquidation model.

Privacy

Private position state is the target. Normal public settlement may expose transfer metadata.

ZK

Proving can be expensive. The implementation therefore separates proving from verification and allows dedicated prover infrastructure.

TEE

Confidentiality depends partly on Vela/Nitro deployment assumptions.

Oracle

Risk safety depends on authenticated/fresh price inputs.

Product rule

Do not add features outside the core state machine before the testnet architecture is stable.

33. Future Extensions

Possible later versions

Multi-collateral

Add ZEN/ETH collateral with per-asset risk parameters.

User-funded liquidity

Allow users to supply ZEN/ETH. This changes reserve, interest and privacy models.

Advanced interest curves

Introduce utilization/kink models after baseline accounting is stable.

Advanced liquidation discovery

Explore more decentralized keeper/candidate mechanisms.

Stronger settlement privacy

Evaluate stealth/one-time destination approaches if supported by the underlying settlement stack.

Advanced ZK

Only add recursion/advanced proving if supported by the selected verifier and shown to improve the system enough to justify complexity.

Upgrade principle

Extensions must be versioned and preserve the V1 invariants.

34. Implementation Roadmap

Phase 0 — Toolchain lock

Pin:

- Vela commit;
- Go;
- TinyGo;
- Node;
- pnpm;
- TypeScript;
- ethers;
- Noir;
- Barretenberg;
- zkVerifyJS;
- contract tooling.

Phase 1 — Vela skeleton

- boot Starter Kit;
- create Noct WASM;
- create thin bridge;
- validate private state persistence.

Phase 2 — State

Implement:

- account;
- cash;
- collateral;
- borrowed balances;
- debt;
- nonce;
- commitments.

Phase 3 — Core accounting

Implement and test:

- deposit;
- supply;
- release collateral;
- cash withdrawal.

Phase 4 — Lending

Implement:

- reserve funding;
- borrow;
- borrowed-asset withdrawal;
- index-based debt model.

Phase 5 — Repay

Freeze exact repayment/interest rules, then implement full/partial repayment.

Phase 6 — Liquidation

Freeze:

- oracle;
- threshold;
- bonus;
- close factor;
- discovery;
- settlement.

Phase 7 — ZK benchmark

Build minimal circuits and benchmark the full path before making ZK mandatory.

Phase 8 — ZK/zkVerify

Integrate the selected verification mode and verify circuit/VK hashes.

Phase 9 — Vela/Horizon integration

Run E2E settlement, recovery and reconciliation tests.

Phase 10 — Security/performance

Run invariant, adversarial, privacy and load tests.

Phase 11 — Testnet

Deploy only after toolchain, proof, settlement and performance evidence are recorded.